

INTELLIGENT DEAD RECKONING

SIH Problem Statement 26168

PLANNING & IMPLEMENTATION PLAN

From Verified Android Foundation to Scientifically Valid Live Dead Reckoning

Purpose of this plan

Convert the current verified repository baseline into an ordered execution program. The plan starts from what is already implemented on GitHub, preserves frozen acquisition/recording/export/replay/map behavior, and places the scientific critical path - frame/calibration correctness, INS, learned correction, fusion and validated GNSS-outage performance - ahead of optional routing or cosmetic expansion.

Repository: SaiChayank/intelligent-dead-reckoning

Baseline: main @ da84645 (da84645fc39f07363a58d3cee035bd196186f11d)

Companion sources: Project Documentation + Master Blueprint

Document status: EXECUTION BASELINE - CURRENT STATE

Document Control

Field	Value
Document	SIH PS26168 - Intelligent Dead Reckoning - Planning & Implementation Plan
Role	Execution sequence, work packages, gates, verification and repository-impact plan
Repository baseline	GitHub main @ da84645fc39f07363a58d3cee035bd196186f11d
Upstream source of truth	SIH_PS26168_Project_Documentation_Current_State + SIH_PS26168_Master_Blueprint_Current_State
Reference structure	PS26145 supplied decision-chain documents; implementation/deployment, verification, demo and audit discipline reused structurally only
Current execution pointer	G4 Frame / Calibration gate is the first scientific blocker; Python session interoperability can proceed in parallel
Change rule	Do not silently alter frozen Prompt 2/3 behavior or contracts. Version incompatible schema changes; raise conflicts before implementation.
Plan date	25 Sep 2026

How to use this file

Execute phases in gate order. Parallel support work is allowed only when it does not bypass a scientific dependency. A phase is not complete because code exists; it exits only when its listed tests and evidence pass. Optional routing/guidance must never displace the core <10% GNSS-outage drift objective.

Plan Map

#	Section
1	Execution principles and status labels
2	Current baseline and immediate gap analysis
3	Critical path and parallel workstreams
4	Phase P0 - Baseline freeze and repository synchronization
5	Phase P1 - Python session interoperability and evaluation harness
6	Phase P2 - Canonical calibration and vehicle-frame closure
7	Phase P3 - Shared navigation core and classical INS
8	Phase P4 - Data/feature pipeline and AI correction
9	Phase P5 - GNSS quality, fusion and recovery
10	Phase P6 - Vehicle constraints and confidence
11	Phase P7 - Map matching and real live-map integration
12	Phase P8 - Offline routing, rerouting and guidance (optional)
13	Phase P9 - Edge engine (secondary)
14	Phase P10 - End-to-end hardening, performance and multi-device validation
15	Phase P11 - SIH demo, release candidate and closure
16	Cross-phase test matrix
17	Repository/file implementation map
18	Git/branch/review workflow
19	Effort and prioritization
20	Risk register and stop conditions
21	Final acceptance gate and freeze checklist

1. Execution Principles and Status Labels

This plan inherits the source hierarchy and frozen decisions from the current-state documentation and master blueprint. Repository evidence determines what is already implemented; scientific claims require executed evidence, not UI appearance.

Label	Execution meaning
FROZEN	Do not redesign unless a proven conflict/defect is documented and approved.
VERIFIED	Code plus executed host/device evidence exists for the stated scope.
IMPLEMENTED	Code exists; acceptance may still be narrower than final target.
BLOCKER	Later dependent work must not be accepted until this gate passes.
CORE	Required for the final dead-reckoning outcome.
HIGH-VALUE	Useful for SIH quality/demo but downstream of core estimator correctness.
OPTIONAL	Implement only after core acceptance path is protected.
FUTURE	Not required for the first mobile acceptance candidate.

- Preserve Prompt 2 acquisition semantics: foreground ownership, permission handling, exact sensor/fix timestamps, bounded queues and explicit diagnostics.
- Preserve Prompt 3 recording/export/replay semantics: downstream recording, exact JSONL/metadata lineage, local-only export, replay_real/replay_simulation, no silent repair of corrupt interior data.
- Keep the offline map as a consumer of NavigationState. Map rendering must never become a second localization/filtering system.
- Never use legacy IO-VNBD frame assumptions, map snapping, cosmetic interpolation or AI output to make an invalid mechanization appear correct.
- No phase may claim physical performance unless it was executed on a defined device/reference scenario with evidence.

2. Current Baseline and Immediate Gap Analysis

Capability	State at baseline	Execution consequence
Android acquisition	VERIFIED	Regression-test only unless a defect is demonstrated.
contracts/v1 + recording/v1	VERIFIED	Treat as immutable v1; incompatible additions require a new version.
Recorder / recovery	VERIFIED	Use as the canonical live evidence pipeline.
Saved sessions / local ZIP export	VERIFIED	Use for reproducible evaluation and sharing; no cloud path.
Kotlin replay	VERIFIED	Use for deterministic Android regression and future engine replay.
Offline Hyderabad MapLibre renderer	VERIFIED enhancement	Retain; later connect to real engine output.
Synthetic GNSS/DR/recovery map demo	VERIFIED demo only	Keep clearly labelled; never count as navigation evidence.
Python session reader/comparison	PARTIAL GAP	Close early so every later physical run is analyzable cross-language.
Frame/calibration solution	BLOCKER	First scientific critical-path task.
Classical INS	EXPERIMENTAL / invalid baseline	Rebuild only after frame/calibration gate.
AI correction	NOT IMPLEMENTED	Blocked by credible baseline and leakage-safe data pipeline.
Fusion / recovery	NOT IMPLEMENTED	Blocked by stable INS state and GNSS policy.
Map matching / live real map	NOT IMPLEMENTED	Blocked by trustworthy NavigationState.
Routing / guidance	OPTIONAL	Do not start before live navigation output is stable.
Edge engine	FUTURE / secondary	After shared-core correctness.

Current execution pointer

The project should not resume at “more map features.” The map demo is already a verified presentation enhancement. The critical-path next gate is canonical calibration / frame correctness. In parallel, close the Python exported-session reader/comparison gap and synchronize stale root documentation.

3. Critical Path and Parallel Workstreams

Critical path

G4 Frame/Calibration -> G5 Classical INS -> G6 AI Correction -> G7 Fusion/Recovery -> G8 Vehicle Constraints/Map Matching -> G9 Live Map -> G10 Final SIH Acceptance. Routing and the 200 Hz edge target remain downstream.

Track	Purpose	Order / dependency
A - Scientific core	Calibration, mechanization, AI aid, fusion, vehicle constraints, map matching	Strict gate order; defines final navigation credibility.
B - Reproducibility	Python session reader, comparison tooling, CI, evidence capture	Can run in parallel with P2; must be ready before large evaluation campaigns.
C - Android product	Engine binding, state UI, live map, diagnostics	Begins after core outputs are stable; map integration after P5/P6.
D - Optional navigation UX	Offline routing, rerouting, turn instructions	Only after P7; never blocks core SIH outcome.
E - Edge target	FOG/high-rate ingestion and ~200 Hz runtime	After shared-core/mobile correctness; secondary acceptance path.

Recommended execution model: one feature branch per phase, with small defect branches inside a phase when necessary. Merge only when that phase reaches its documented gate; keep main as the latest verified checkpoint.

4. Phase P0 - Baseline Freeze and Repository Synchronization

Field	Plan
Current state	READY / support task
Objective	Create an unambiguous starting checkpoint for future algorithm work without changing verified runtime behavior.
Entry condition	Current main at da84645; Project Documentation and Master Blueprint available.
Estimated engineering effort	0.5-1 person-day

4.1 Implementation work

1. Update stale current-state wording in the root README so it no longer says recording/replay/maps are missing. Preserve historical readiness reports as historical evidence rather than rewriting them.
2. Add a concise CURRENT_STATUS.md or equivalent that links the verified acquisition/recording/export/replay/map evidence and states the G4 blocker.
3. Run the complete existing Python + Android host test suites and record the exact baseline counts before algorithmic changes.
4. Create an immutable Git tag or release-style checkpoint for the current verified foundation after review.
5. Capture dependency versions, Android build identifiers and current offline-map asset manifest in the baseline note.

4.2 Expected repository impact

Area	Files / modules
Docs	README.md; optional docs/CURRENT_STATUS.md; companion documents under docs/ if committed.
Tests	No production test changes expected unless stale paths are corrected.
Git	Tag suggestion: idr-foundation-map-da84645 or date-based equivalent after owner approval.

4.3 Verification gate

- Python unit suite passes without raw-data mutation.
- Gradle testDebugUnitTest, lintDebug and assembleDebug pass.
- Existing Android instrumentation remains unchanged; rerun if production code is touched.

- `git diff --check` passes and no generated recordings/APKs/artifacts are staged.

Definition of Done

Current repository status is synchronized with evidence; main remains behaviorally identical to the verified foundation; a future contributor can identify the exact starting gate without reading stale historical claims.

4.4 Main risks / controls

Risk	Control
Accidentally rewriting historical evidence	Update only current-status summaries; do not edit old dated reports to pretend later features existed.
Tagging an unreviewed working tree	Require clean status and reviewed commit before tagging.

5. Phase P1 - Python Session Interoperability and Evaluation Harness

Field	Plan
Current state	PARTIAL GAP / parallel support
Objective	Create a reusable Python equivalent of the Kotlin session reader/validator so every later device recording can be analyzed reproducibly.
Entry condition	Frozen recording v1 contract and verified ZIP export are available.
Estimated engineering effort	1-2 person-days

5.1 Implementation work

1. Implement a streaming Python session API for an unpacked directory and the canonical ZIP package. Reuse `contracts.v1` and `contracts.recording.v1` codecs; do not duplicate wire validation.
2. Validate metadata/session identity, versions, counts, source lineage, UTF-8, corrupt interior rows and the narrow recovered-session policy. Preserve Int64 timestamps exactly.
3. Implement deterministic summaries: record/channel counts, sensor rates, GNSS providers/null counts, first/last timestamps, duration and diagnostics counts.
4. Implement Kotlin/Python comparison fixtures and a CLI for `validate/summary/compare`. Exact-match IDs/timestamps/nulls/enums/integers; documented tolerances only for binary64 sensor fields.
5. Add ZIP safety checks: only canonical entries, no traversal/absolute paths/duplicates, bounded metadata and streaming measurements.
6. Use one real exported session and one recovered session as read-only acceptance artifacts outside Git; record hashes rather than committing user data.

5.2 Expected repository impact

Area	Files / modules
Python package	Suggested: <code>recordings/</code> or <code>evaluation/session_io/</code> with <code>reader.py</code> , <code>summary.py</code> , <code>compare.py</code> , <code>__main__.py</code> .
Tests	<code>tests/test_session_reader.py</code> ; <code>tests/test_session_interop.py</code> ; shared golden fixtures only.
Docs	<code>docs/session_interoperability.md</code> or extension to <code>mobile/REPLAY.md</code> with Python boundary.

5.3 Verification gate

- Completed and zero-record sessions validate.
- Recovered/incomplete session behavior matches the frozen contract.
- Corrupt interior lines, unsupported versions, invalid UTF-8 and count mismatch fail explicitly.
- Timestamps above 2^{53} remain exact Python ints.
- Canonical ZIP entry/path safety tests pass.

- Kotlin/Python fixture summaries match exactly where specified.

Definition of Done

Python can validate and summarize the exact Android-exported session without reinterpretation or IO-VNBD assumptions, enabling repeatable algorithm evaluation.

5.4 Main risks / controls

Risk	Control
Creating a second permissive parser	All measurement decoding must delegate to the frozen contract codec.
Treating session readback as IO-VNBD validation	Document explicitly that fresh Android recordings do not certify historical dataset export semantics.

6. Phase P2 - Canonical Calibration and Vehicle-Frame Closure

Field	Plan
Current state	OPEN / BLOCKER (G4)
Objective	Establish a causal, physically defensible device-to-vehicle transform and bias procedure for live Android measurements.
Entry condition	Verified raw Android acquisition; physical device axes already sanity-checked; legacy IO-VNBD export semantics remain unresolved.
Estimated engineering effort	3-5 person-days, evidence-dependent

6.1 Implementation work

1. Define the deployable calibration protocol before code: phone mounting assumptions, stationary duration, required maneuvers, rejection criteria, remount/session invalidation and which sensors are mandatory/optional.
2. Use fresh canonical Android-device-frame captures for deployable calibration. Keep IO-VNBD only for bounded research/reference analyses where field semantics are supported.
3. Estimate gyro bias causally from stationary windows; define accelerometer bias treatment and gravity use without double-subtraction. Magnetometer may assist heading only if interference tests support it.
4. Estimate/validate `q_vehicle_from_device` as a proper rotation (determinant +1); no reflection/sign-permutation shortcuts. Freeze FLU vehicle and ENU navigation conventions.
5. Emit a real CalibrationResult record with ID, validity state, transform/biases and explicit confidence semantics. Do not infer a valid calibration from UI orientation alone.
6. Build a calibration capture/diagnostic workflow that records evidence but does not alter the acquisition event contract.
7. Run stationary, straight-line, turns and braking/acceleration maneuvers with a defined external/reference position source where available. Validate stability across repeated mountings or explicitly scope the mount as fixed.

6.2 Expected repository impact

Area	Files / modules
Core math	New UI-independent calibration/frame module under the future navigation core; Python reference math may mirror it.
Android integration	Calibration coordinator/UI only after algorithm contract is stable; existing acquisition remains untouched.
Contracts	Use existing CalibrationResult if sufficient; any semantic/field change requires a new contract version, not mutation of v1.
Reports	reports/calibration_protocol.md; reports/calibration_acceptance_<date>.md/json.
Tests	Synthetic proper-rotation, six-face, bias, remount invalidation and causal-window tests.

6.3 Verification gate

- Synthetic rotation/quaternion tests pass within frozen tolerances.
- Stationary bias estimate is causal and reproducible.
- No reflected transform is accepted.
- Calibration invalidates on new/remounted session as designed.
- Repeated real maneuvers meet the predeclared alignment/error gates; failures are retained, not tuned away.
- CalibrationResult is reproducible from recorded inputs and can be replayed.

Definition of Done

G4 PASS: an approved calibration/frame procedure exists for live Android device-frame measurements, with real evidence and a versioned output usable by mechanization. Only then may production INS begin.

6.4 Main risks / controls

Risk	Control
Mounting dependence	Scope MVP to a fixed mount if necessary; make remount invalidation explicit.
Magnetic interference	Do not make magnetometer mandatory unless vehicle tests support it.
Legacy dataset temptation	Never feed world-like IO-VNBD gravity/acceleration into a device-frame calibration path as if raw.
Insufficient reference truth	Use controlled geometry/GNSS/reference runs; if no adequate reference exists, mark G4 not passed rather than relaxing the gate.

7. Phase P3 - Shared Navigation Core and Classical INS

Field	Plan
Current state	BLOCKED BY G4 (G5)
Objective	Implement a deterministic, UI-independent mechanization engine and establish a credible pre-AI baseline.
Entry condition	G4 calibration/frame acceptance and stable v1 input semantics.
Estimated engineering effort	4-6 person-days

7.1 Implementation work

1. Implement navigation state, frame transforms, quaternion propagation, specific-force/gravity handling, velocity/position integration and measured dt from event timestamps.
2. Implement strict initialization_mode handling: deployable mode uses only causally available calibration/GNSS; evaluation mode may use reference initialization only when explicitly labelled.
3. Handle gaps/out-of-order/late inputs using bounded buffering and explicit diagnostics; never integrate across a forbidden gap silently.
4. Implement NavigationEngine behind the existing interface or a versioned compatible interface if evidence proves v1 insufficient. Keep Android/Compose/MapLibre dependencies out of the core.
5. Emit NavigationState at approximately 10 Hz on mobile while ingesting IMU at native rates. Keep sensor cadence separate from UI cadence.
6. Build a Python reference implementation or deterministic replay oracle for parity/evaluation where practical.
7. Run the complete synthetic mechanization suite before interpreting real trajectories: stationary, constant velocity, constant acceleration, straight motion, constant-rate turn/circle, gyro bias, accelerometer bias, irregular dt, dropped sample/gap and coordinate-frame cases.
8. Run short held-out real/reference segments and publish baseline errors. Do not proceed to AI if the classical engine is still catastrophically wrong for frame/model reasons.

7.2 Expected repository impact

Area	Files / modules
Core	Suggested core/navigation/{state,frames,mechanization,initialization,buffering}.kt or equivalent UI-independent module.
Python reference	core_py/ or training/reference_navigation.py only if it shares equations/tests rather than becoming a divergent production path.
Android binding	mobile/.../navigation/NavigationEngineImpl.kt after core tests pass.
Tests	Synthetic mechanization suite + Kotlin/Python parity fixtures.
Reports	reports/classical_ins_baseline_<date>.md/json with per-scenario metrics.

7.3 Verification gate

- All synthetic truth cases pass declared numeric tolerances.
- Initialization never consumes future/reference data in deployable mode.
- Output timestamps/session/source lineage are exact and ordered according to the engine policy.
- No unbounded state/history grows with trip length.
- Classical real-data/reference baseline is physically credible enough to attribute remaining error to drift/model limitations rather than gross frame bugs.

Definition of Done

G5 PASS: a credible classical dead-reckoning engine exists with reproducible synthetic and real-reference evidence. AI is allowed to optimize a valid baseline rather than compensate for undefined frames.

7.4 Main risks / controls

Risk	Control
Reusing historical INS code without fixing assumptions	Treat training/ins_mechanization.py as historical/diagnostic unless its assumptions are explicitly corrected.
Nominal-dt shortcuts	Use measured monotonic dt and explicit gap policy.
UI-coupled core	No Compose/MapLibre/Activity dependencies in navigation math.
Evaluation leakage	Keep evaluation-only initialization visibly separate from deployable mode.

8. Phase P4 - Data/Feature Pipeline and AI Correction

Field	Plan
Current state	BLOCKED BY G5 (G6)
Objective	Train a causal learned motion aid/error-correction model that demonstrates held-out improvement over the credible classical INS baseline and can run on device.
Entry condition	G5 pass; feature/label semantics and evaluation splits frozen before model tuning.
Estimated engineering effort	5-8 person-days, dataset-dependent

8.1 Implementation work

1. Define the ML task precisely before training: e.g., speed/velocity aid, bias/residual correction or another bounded motion aid. Do not train an opaque direct lat/lon predictor as a shortcut around mechanization.
2. Build causal windows/features from the same definitions used at inference. Document sensor rate handling, missing channels, normalization and warm-up behavior.
3. Freeze route/driver/session-based train/validation/test splits to prevent temporal and identity leakage. Keep final held-out sequences untouched during feature/model selection.

4. Use IO-VNBD only for features/labels whose semantics are supported; prefer fresh canonical captures for deployable frame-dependent features. Record all exclusions.
5. Establish simple baselines first (linear/regression/tree model as appropriate), then compare a compact temporal/MLP/TCN candidate only if it materially improves generalization and is exportable.
6. Calibrate/normalize model outputs and define failure/fallback behavior. No online self-training in the MVP.
7. Export a versioned ONNX or TFLite artifact only after Android runtime benchmarking; include feature schema, normalization constants, training provenance and checksum.
8. Add train/inference parity tests using the same feature code or a single generated specification so the Android runtime cannot silently drift from Python training.

8.2 Expected repository impact

Area	Files / modules
Training	training/ml/ data preparation, feature extraction, model training/evaluation, export.
Model registry	models/manifest or metadata tracked in Git; large/binary model artifact handling follows .gitignore/release policy.
Android inference	mobile/.../navigation/ai/ wrapper after export format selection.
Tests	Leakage, split determinism, feature parity, export parity, fallback and latency tests.
Reports	reports/ai_model_card_<version>.md; reports/ai_evaluation_<version>.json/md.

8.3 Verification gate

- Frozen split manifest is reproducible and leakage checks pass.
- Classical baseline and AI-assisted result are evaluated on the same untouched outages.
- AI shows statistically/operationally meaningful held-out improvement, not only training-score gain.
- Exported runtime matches Python reference within documented tolerances.
- On-device inference meets the latency/power budget without changing raw measurement semantics.
- Model provenance, feature schema and checksum are recorded.

Definition of Done

G6 PASS: a versioned learned aid improves the valid classical baseline on held-out data and reproduces on Android. If it does not, keep classical/fusion path and do not claim AI benefit.

8.4 Main risks / controls

Risk	Control
Data leakage	Split by whole sessions/routes/drivers before window generation where needed; audit overlap.
Model hides frame errors	G5 is a hard prerequisite; AI cannot waive mechanization acceptance.
Over-complex network	Prefer the smallest model that passes held-out and runtime gates.
Export mismatch	Parity-test exact preprocessing and inference before accepting mobile results.

9. Phase P5 - GNSS Quality, Fusion and Recovery

Field	Plan
Current state	BLOCKED BY G5/G6 INPUTS (G7)
Objective	Create the real GNSS-available -> DR -> recovery behavior and calibrated uncertainty needed by the final system.
Entry condition	Credible INS state; AI aid accepted or explicitly disabled; GNSS quality inputs defined.
Estimated engineering effort	4-6 person-days

9.1 Implementation work

1. Freeze a GNSS quality policy using fix age, provider, accuracy, satellites and reason codes. Thresholds must be measured/justified; do not promote synthetic examples to real policy.
2. Implement the first production fusion candidate with explicit state/covariance definitions, process/measurement noise and innovation gating. Use EKF-first execution unless a documented nonlinearity/observability test justifies escalating to UKF.
3. During GOOD GNSS, update the estimator; during stale/denied/unavailable states, continue inertial/AI propagation without silently reusing an old fix as current.
4. On GNSS recovery, gate outliers and reintroduce updates through estimator innovation/covariance. Any map marker smoothing is display-only and must not alter estimator truth.
5. Emit calibrated Confidence records only after empirical coverage tests. Until then use UNVALIDATED/UNAVAILABLE, not fabricated probability/95% radius.
6. Add deterministic dropout injection to replay: exact outage windows, degraded accuracy, stale fixes, delayed recovery and gross-outlier cases.
7. Evaluate final/RMS position error, drift/distance, velocity/heading error, innovation behavior and recovery correction magnitude on held-out trajectories.

9.2 Expected repository impact

Area	Files / modules
Core	navigation/fusion, gnss_policy, confidence modules.
Contract	If a distinct GNSS/DR/FUSED/RECOVERY mode is required for downstream UI, create contracts/v2 or another explicit version; do not mutate strict v1.
Replay/evaluation	Outage injector consuming recorded sessions without rewriting originals.
Tests	Filter linearization/numerics, deterministic dropout, stale/outlier/recovery cases, covariance sanity.
Reports	reports/fusion_recovery_<date>.md/json; GNSS quality policy document.

9.3 Verification gate

- Known synthetic estimator cases converge and remain numerically stable.
- GNSS denial produces continuous DR output without fresh GNSS updates.
- Stale/repeated fix handling is explicit.
- Recovery rejects outliers and rejoins GNSS without uncontrolled estimator jump.
- Confidence remains honestly labelled and empirically checked.
- Held-out outage metrics are reproducible from recorded sessions.

Definition of Done

G7 PASS: the system has a validated estimator state machine for healthy GNSS, degraded/denied propagation and recovery, with evidence separate from display animation.

9.4 Main risks / controls

Risk	Control
Overconfident covariance	Evaluate coverage; keep confidence unvalidated until evidence supports calibration.
Old GNSS reused as new	Track fix age and event timestamps; no silent hold-as-current.
Cosmetic recovery	Measure estimator innovation/correction; display interpolation may not conceal it.
Contract-mode ambiguity	Version the schema rather than inferring UI mode from unrelated fields.

10. Phase P6 - Vehicle Constraints and Confidence Hardening

Field	Plan
Current state	BLOCKED BY G7 STATE STABILITY (part of G8)
Objective	Reduce physically impossible vehicle motion and improve uncertainty behavior without forcing assumptions outside their valid operating envelope.
Entry condition	Fusion state and covariance semantics are stable and replayable.
Estimated engineering effort	2-4 person-days

10.1 Implementation work

1. Implement non-holonomic constraints (low lateral/vertical vehicle velocity) behind explicit gating based on motion, speed, road context or confidence; do not apply unconditionally.
2. Add zero-velocity/stationary detection only if false-positive behavior is measured. Avoid treating traffic-stop GNSS noise as motion or vice versa.
3. Add physically plausible speed/acceleration/yaw-rate sanity bounds as diagnostics or soft constraints, not hidden clipping.
4. Compare estimator performance with constraints disabled/enabled across straight, turn, braking, bumps and low-speed maneuvers.
5. Finalize confidence semantics only after evaluating empirical error coverage by mode and outage duration.

10.2 Expected repository impact

Area	Files / modules
Core	navigation/constraints, motion_state, confidence calibration.
Tests	Constraint-gating and pathological-motion fixtures.
Reports	reports/constraint_ablation_<date>.md/json; confidence calibration plots/data.

10.3 Verification gate

- Constraints improve or preserve held-out performance without creating turn/low-speed failure modes.
- Every applied constraint is traceable in diagnostics/evaluation.
- Confidence/uncertainty behavior grows appropriately with outage duration and is not cosmetically fixed.

Definition of Done

Constraint set is accepted as a measurable estimator aid and is safe to feed into map matching. Raw pre-constraint/fused metrics remain available for ablation.

10.4 Main risks / controls

Risk	Control
Constraint overreach	Gate by motion state and test uncommon maneuvers.
Hidden clipping	Expose intervention diagnostics; keep ablation path.
Uncalibrated confidence	Do not label 95% uncertainty as calibrated until coverage testing supports it.

11. Phase P7 - Map Matching and Real Live-Map Integration

Field	Plan
Current state	BLOCKED BY G7/G8 (G8/G9)
Objective	Connect trustworthy real navigation output to the already-verified offline map, add road matching without losing raw-state traceability, and demonstrate genuine GNSS/DR/recovery movement.

Field	Plan
Entry condition	Fusion/constraints stable; navigation mode contract decision complete; offline MapLibre renderer already verified.
Estimated engineering effort	3-5 person-days

11.1 Implementation work

1. Build a local road-geometry/graph representation for the Hyderabad demo region separate from the MBTiles renderer data. Define licensing/provenance and deterministic preprocessing.
2. Implement a probabilistic map matcher (simple candidate scoring first; HMM/Viterbi if ambiguity/outage tests justify it) using fused pose, heading and uncertainty. Never feed map-matched coordinates back into the estimator unless explicitly designed/versioned later.
3. Preserve both fused/raw and matched positions in evaluation output so map matching cannot hide estimator error.
4. Bind the real NavigationEngine output stream to NavigationPresentation/MapPresentation. Remove no synthetic demo; keep it as a separate labelled demonstration mode.
5. Show real current vehicle marker, heading, travelled path, confidence radius and explicit navigation mode. Update at the bounded display cadence, not IMU callback rate.
6. During GNSS loss, marker movement must come from NavigationState/DR. On recovery, map shows estimator convergence; display interpolation is allowed only for visual smoothness and must not modify state or metrics.
7. Add map follow/recenter behavior that does not block lifecycle, recording, replay or offline operation.
8. Run tunnel/covered-parking/controlled-dropout tests and road-ambiguity cases. Capture raw fused vs matched vs reference metrics.

11.2 Expected repository impact

Area	Files / modules
Map matcher	navigation/map_matching or separate core module; local graph assets/preprocessor.
Android map binding	mobile/.../map/ real NavigationState collector; reuse existing renderer/presentation abstractions.
Contracts	Versioned navigation mode and optionally matched-pose event/field if required; strict v1 remains intact.
Tests	Road candidate/heading/branch ambiguity, GNSS outage continuity, lifecycle, replay/live source isolation.
Reports	reports/live_map_acceptance_<date>.md; map-matching ablation results.

11.3 Verification gate

- Live map consumes real engine output, not Android Location directly and not synthetic controller output.
- GNSS denial continues marker/trail motion from DR; recovery reflects estimator behavior.
- Raw fused and road-matched positions remain separately inspectable.
- Map matching improves road plausibility without worsening reference error beyond defined tolerance.
- Offline map still renders with no INTERNET permission/network fallback.
- Physical target-device test passes with recording/replay unaffected.

Definition of Done

G8/G9 PASS: real GNSS/DR/recovery navigation drives the offline map with traceable map matching and no second localization path.

11.4 Main risks / controls

Risk	Control
Map matching masks estimator drift	Always report raw fused error beside matched error.
Renderer starts reading Location API	Prohibit direct location ownership in map module; consume navigation records only.

Risk	Control
Scope creep into routing	Keep routing P8 optional and downstream.
Regional graph/storage growth	Use bounded Hyderabad demo-region data first.

12. Phase P8 - Offline Routing, Rerouting and Guidance (Optional)

Field	Plan
Current state	OPTIONAL AFTER G9
Objective	Add a zero-runtime-API-cost route planner and basic turn guidance without changing localization.
Entry condition	Real live map integration passes; local routable graph available or can be generated reproducibly.
Estimated engineering effort	3-6 person-days; optional

12.1 Implementation work

1. Evaluate one offline Android-suitable routing stack (e.g., GraphHopper/BRouter-class approach) against APK/storage/memory/licensing constraints. Keep this as a bounded decision spike, not a multi-engine project.
2. Generate a routable local graph from OSM-derived regional data; do not treat MBTiles as the routing graph.
3. Implement origin/destination selection, route polyline, route progress and off-route detection. Position input is the accepted matched/fused navigation state.
4. Add rerouting only when off-route persistence exceeds a defined threshold; never reroute because of transient estimator noise alone.
5. Generate basic maneuver/turn instructions locally. Keep voice guidance optional unless it is essentially free after route instructions exist.
6. Document attribution/license obligations and regional storage footprint.

12.2 Expected repository impact

Area	Files / modules
Routing	New routing/ module + graph-build tool/assets.
UI	Destination/route summary/turn card integrated with map screen.
Tests	Known-route shortest path, no-path, off-route/reroute, graph corruption, offline-only execution.
Docs	offline routing data-build and licensing guide.

12.3 Verification gate

- Routing works with radios/network unavailable.
- No paid/API key dependency is required at runtime.
- Route planning does not feed back into estimator state.
- Rerouting is stable under small localization noise.
- Storage/memory remains acceptable for the bounded demo region.

Definition of Done

Optional feature accepted for prototype UX. Failure or deferral of this phase does not invalidate the core dead-reckoning solution.

12.4 Main risks / controls

Risk	Control
Storage blow-up	Use regional graph and measure actual installed size.
Licensing complexity	Preserve OSM/engine notices and data provenance.
Reroute thrash	Use persistence/hysteresis based on matched-state confidence.
Core schedule risk	Drop this phase before delaying G10 core acceptance.

13. Phase P9 - Edge Engine (Secondary)

Field	Plan
Current state	FUTURE / G11
Objective	Reuse the accepted navigation contracts/core with higher-rate FOG-class IMU input and target roughly 200 Hz processing.
Entry condition	Mobile shared core stable; model export/runtime format settled; edge hardware/input format available.
Estimated engineering effort	3-5 person-days after mobile acceptance

13.1 Implementation work

1. Define an edge input adapter that maps FOG/GNSS data into the same canonical measurement semantics without Android dependencies.
2. Reuse frame, INS, fusion, constraints and model inference logic where language/runtime permits; otherwise require numeric parity fixtures rather than an independent algorithm rewrite.
3. Benchmark sustained ~200 Hz input, output cadence, latency, CPU/memory and drop behavior.
4. Preserve recording/replay-equivalent evidence on edge so mobile/edge results can be compared.
5. Do not expand edge work until mobile G10 acceptance is secure unless the SIH submission explicitly requires it.

13.2 Expected repository impact

Area	Files / modules
Edge	edge/ runtime, adapters, benchmark harness.
Shared fixtures	Golden navigation sequences consumed by mobile/core and edge.
Reports	reports/edge_benchmark_<date>.md/json.

13.3 Verification gate

- Canonical input/output parity tests pass.
- Sustained target-rate benchmark has bounded memory/queues and no silent drops.
- Navigation accuracy does not regress relative to the shared algorithm on identical input.

Definition of Done

G11 evidence exists for the secondary target without creating a forked algorithm architecture.

13.4 Main risks / controls

Risk	Control
Parallel algorithm divergence	Use shared fixtures/contracts and parity gates.
Hardware unavailable	Keep phase future; do not fabricate throughput claims.

14. Phase P10 - End-to-End Hardening, Performance and Multi-Device Validation

Field	Plan
Current state	AFTER G9; REQUIRED BEFORE G10
Objective	Prove the integrated mobile system remains bounded, offline, reproducible and usable beyond isolated algorithm tests.
Entry condition	Core estimator + live map integrated; recording/replay still passing.
Estimated engineering effort	3-5 person-days

14.1 Implementation work

1. Run long foreground drives/recordings with the navigation engine active; profile PSS/RSS, CPU, thermal state, battery draw, queue depths and missed navigation outputs.
2. Benchmark input-to-NavState latency and approximately 10 Hz output cadence; freeze explicit p50/p95/p99 acceptance thresholds after the first measured run.
3. Run complete offline/airplane-mode acceptance: acquisition, navigation, map, recording and replay must not require network access.
4. Execute controlled GNSS-denied scenarios: tunnel/covered parking where safe, plus deterministic replay dropouts so tests are reproducible.
5. Add a second Android device/API level if available to detect assumptions tied to the OnePlus 12R.
6. Run destructive storage/map-pack fault tests only on a disposable installation, never on retained user recordings.
7. Add CI for host Python + Gradle unit/lint/build where feasible; device tests remain explicit hardware jobs/manual evidence if no stable device farm exists.
8. Perform security/privacy review: permissions, no background service, no internet/upload, exported-file handling, ZIP safety, path leakage, model/map asset integrity.

14.2 Expected repository impact

Area	Files / modules
Benchmarks	mobile/benchmark or instrumentation harness; Python analysis scripts for exported logs.
CI	GitHub Actions for host-only tests/build if dependency caching/reproducibility is acceptable.
Reports	reports/end_to_end_acceptance_<date>.md; performance JSON; device matrix.
Docs	Update VERIFICATION.md and current status only after evidence exists.

14.3 Verification gate

- No unbounded memory or queue growth across sustained integrated runs.
- Navigation output cadence/latency targets are measured and met or explicitly revised with evidence.
- Offline operation passes with no mandatory network permission/request.
- All frozen acquisition/recording/export/replay regressions pass.
- Second-device smoke acceptance completed if hardware is available.
- Security/privacy checklist has no blocker.

Definition of Done

Integrated mobile release candidate is stable enough for final SIH performance evaluation and demo rehearsal.

14.4 Main risks / controls

Risk	Control
Performance optimization breaks correctness	Profile first; optimize only after reproducible bottleneck evidence.
Single-device assumptions	Add at least one second device/API smoke test before broad compatibility claims.
Destructive tests damage recordings	Use disposable install/user data only.
CI gives false physical confidence	Keep hardware acceptance separate from host CI.

15. Phase P11 - SIH Demo, Release Candidate and Closure

Field	Plan
Current state	FINAL G10
Objective	Package a truthful, reproducible prototype demonstration

Field	Plan
	centered on real GNSS-outage continuity and measured drift.
Entry condition	P10 pass; at least one held-out scenario approaches or meets the frozen target; fallback demo path prepared.
Estimated engineering effort	2-4 person-days plus rehearsal

15.1 Implementation work

1. Freeze a release-candidate commit and build ID. No feature changes after rehearsal unless fixing a blocker; rerun affected gates after any fix.
2. Prepare one primary real route and one deterministic recorded/replay fallback. Keep the synthetic map demo as a clearly labelled visualization fallback, not proof of AI/DR.
3. Demonstrate real acquisition, navigation, controlled GNSS outage, DR continuation, uncertainty growth, GNSS recovery, drift metric, recording finalization, local export and deterministic replay.
4. Present reference vs estimate metrics for the outage: travelled distance, final error, RMS error and drift/distance. State clearly whether <10% is achieved and on what population/scenarios.
5. If routing/guidance exists, show it only after core navigation proof. If not, omit without apology; it is optional.
6. Prepare a compact judge question matrix: why AI, why no OBD/CAN, offline operation, calibration, failure modes, map matching, dataset limits, battery and privacy.
7. Update final Project Documentation, Master Blueprint and this Plan only with executed results; preserve historical sections and claim boundaries.

15.2 Expected repository impact

Area	Files / modules
Release	Tagged APK/build; model/map/checksum manifest; evidence bundle.
Demo assets	Recorded fallback session, comparison plots/tables, screenshots and scripted operator checklist.
Docs	Final evidence report, README current status, architecture diagram, model card, test matrix.
Git	Release branch/tag only after clean reviewed commit.

15.3 Verification gate

- Primary live demo rehearsal passes end-to-end.
- Fallback replay works completely offline from a known exported session.
- No synthetic/experimental result is mislabeled as measured production navigation.
- All critical metrics and build/model versions are traceable.
- Final repository status is clean; generated recordings/APKs/raw data remain out of Git.

Definition of Done

G10 PASS / final prototype acceptance: the mobile system demonstrates real, on-device GNSS-outage continuity with reproducible evidence and explicitly reports whether the <10% target is met.

15.4 Main risks / controls

Risk	Control
Venue GNSS/tunnel conditions unavailable	Use deterministic replay dropout as the scientific fallback; live demo remains bonus.
Last-minute feature changes	Freeze RC and rerun only blocker fixes.
Overclaiming target attainment	Always state scenario count/population and whether result is target achievement or partial progress.

16. Cross-Phase Test Matrix

Layer	Representative tests	When mandatory
Contract	Strict codec, versions, exact Int64, nulls, source/session membership	Every contract/schema change; all phases
Acquisition regression	Permissions, rates, lifecycle, bounded queue, listener cleanup	Any Android lifecycle/source integration change
Recording/export/replay	Start/stop/recovery, exact ZIP, read-only replay, source mapping	Any engine/UI integration touching session ownership
Calibration math	Quaternion/proper rotation, bias, causal windows, remount invalidation	P2 and all calibration changes
INS synthetic truth	Stationary, CV, CA, circle/turn, bias, irregular dt, gaps	P3; must stay green thereafter
ML	Split leakage, feature parity, held-out improvement, export parity	P4 and model updates
Fusion	Quality states, dropout, stale/outlier, covariance, recovery	P5+
Constraints	Gating/ablation/pathological motion	P6+
Map matching	Road candidates, ambiguity, raw vs matched error	P7+
Live map	Engine-only source, outage movement, recovery, offline rendering	P7+ physical acceptance
Routing	Offline shortest path/no-path/reroute	P8 only
Performance	Cadence, latency, memory, battery/thermal, queues	P10 and RC
Security/privacy	Permissions, offline/no upload, path/ZIP/model/map integrity	P10 and RC
Demo regression	Full live + replay fallback script	P11 after every RC fix

17. Repository / File Implementation Map

The exact package layout may be refined during implementation, but responsibility boundaries should remain stable. New code should be added rather than forcing navigation logic into acquisition, recorder, replay or map-renderer classes.

Area	Current / target responsibility	Implementation direction
contracts/v1	Frozen measurement/navigation exchange	Keep unchanged unless bug; new incompatible semantics go to a new version.
contracts/recording/v1	Frozen recording/session contract	Keep canonical; Python session layer consumes it.
mobile/.../acquisition	Verified hardware input	No navigation/filtering/disk logic added.
mobile/.../recording	Verified downstream recorder	No estimator logic added.
mobile/.../replay	Verified typed replay	Later may feed engine through an explicit evaluation adapter, not by altering timestamps.
mobile/.../map	Verified renderer/presentation	Consume real NavigationState after P7; no raw GPS ownership.
core/navigation or equivalent	Future shared navigation math/state	Calibration, frames, INS, fusion, constraints, matching abstractions.
training/	Offline data science/evaluation	Dataset audit, training, model selection/export, reference evaluation.
recordings/evaluation	Future Python session reader/comparison	Canonical Android recording analysis.
models/	Versioned model metadata/artifacts	Binary policy per repo; always checksum/provenance.
edge/	Secondary high-rate runtime	After shared-core/mobile stability.
reports/	Executed evidence	Never write target claims as verified results.
docs/	Current design/submission documents	Synchronize at frozen checkpoints only.

18. Git, Branch and Review Workflow

Stage	Recommended practice
Start	main must equal the latest verified checkpoint; create one phase branch.

Stage	Recommended practice
Implementation	Small commits scoped to the phase; do not mix formatting/raw-data/generated artifacts.
Before acceptance	Run phase tests + inherited regression suite; inspect git status/diff/diff --check.
Physical gate	Keep branch until required device evidence passes; do not mark READY from compile/tests alone.
Merge	Fast-forward or reviewed merge to main only after gate passes.
Push	Push main as the latest verified stable state; feature branch may also be pushed for backup/review.
Tag	Tag only meaningful frozen checkpoints/RCs after a clean working tree.
Failure	If a phase fails, keep evidence and fix on the same branch or a tightly-scoped defect branch; do not bypass the gate.

Suggested branch sequence

p0-baseline-sync -> p1-python-session-interop (parallel) -> p2-calibration-frame -> p3-classical-ins -> p4-ai-correction -> p5-fusion-recovery -> p6-vehicle-constraints -> p7-live-map-matching -> optional p8-offline-routing -> p10-hardening -> p11-release-candidate. Names are suggestions; gate order is the important part.

19. Effort and Prioritization

Priority	Phases	Reason
P0 - Must protect immediately	P0, P1, P2, P3	Status clarity, reproducibility and physically correct baseline.
P1 - Core SIH outcome	P4, P5, P6, P7	AI-enhanced DR, recovery and real map proof.
P2 - Product hardening	P10, P11	Performance, reliability, demo and truthful acceptance evidence.
P3 - Optional polish	P8	Offline routing/rerouting/turn guidance; valuable but not localization.
P4 - Secondary future	P9	~200 Hz FOG/edge target after mobile core is stable.

Serial effort ranges in this plan are engineering estimates for planning, not promises. Evidence-dependent calibration and model work can expand. Protect the scientific critical path by dropping P8/P9 before weakening P2-P7 acceptance.

Phase	Approx. effort	Can overlap?
P0	0.5-1 d	With P1 setup
P1	1-2 d	Yes; parallel to P2
P2	3-5 d	Critical path
P3	4-6 d	After P2
P4	5-8 d	Feature/data work can start late P3, acceptance cannot
P5	4-6 d	After P3; integrate accepted model when ready
P6	2-4 d	After stable P5
P7	3-5 d	After P5/P6
P8	3-6 d	Optional after P7
P9	3-5 d	Secondary after mobile core
P10	3-5 d	After integrated core
P11	2-4 d + rehearsal	After P10

20. Risk Register and Stop Conditions

Risk / trigger	Severity	Required response
Frame/calibration cannot be validated on fresh canonical data	BLOCKER	Do not start production INS/AI acceptance. Improve capture/reference protocol; retain NO-GO.
Classical INS remains grossly wrong after G4	BLOCKER	Debug equations/timing/initialization before training AI.
AI does not beat classical baseline on held-out data	HIGH	Keep AI out of final claim; continue with classical+fusion or revise task/data transparently.
Fusion recovery produces large unstable jumps	HIGH	Fix quality/gating/noise model; do not hide with UI interpolation.
Map matching improves visual road fit but worsens raw reference error	HIGH	Report both; tune/disable matching. Never use matched trace as sole accuracy metric.
Contract needs GNSS/DR/FUSED mode	MEDIUM	Create versioned contract; v1 remains frozen.
Routing threatens schedule	MEDIUM	Defer P8 immediately; core target has priority.
Battery/thermal unacceptable	MEDIUM	Profile; reduce model/update cadence or optimize implementation without lowering correctness gates.
Only one phone tested	MEDIUM	Limit claim to tested device or add second-device smoke.
Demo environment lacks real outage	MEDIUM	Use deterministic replay-dropout evidence; keep live demonstration separate.
Historical docs conflict with repository	MEDIUM	Current evidence/blueprint wins for status; preserve historical documents as dated records.

Hard stop rule

If G4 or G5 fails, do not move the final-system claim forward by adding AI, map matching, routing or more sophisticated UI. The correct project action is to fix the scientific blocker or report the limitation.

21. Final Acceptance Gate and Freeze Checklist

Gate item	Acceptance evidence
Canonical frame/calibration	Approved causal calibration with real repeatability evidence.
Classical INS	Synthetic truth suite passes; real baseline physically credible.
AI enhancement	Held-out improvement and Android inference parity; model provenance/checksum.
GNSS -> DR -> recovery	Measured state transitions, outage propagation and stable recovery.
Drift target	<10% travelled-distance drift on the explicitly defined held-out outage acceptance set, or an honest documented miss.
Confidence	Uncertainty semantics calibrated/validated or explicitly marked unvalidated.
Map matching	Raw and matched traces both available; no hidden feedback into estimator.
Live offline map	Real engine output drives marker/heading/trail/mode/confidence without network.
Recording/evaluation	Same live run can be finalized, exported, replayed and validated cross-language.
Performance	Approx. 10 Hz output, bounded latency/memory/queues; battery/thermal evidence.
Security/privacy	No mandatory internet/cloud, no broad storage permission, local export only, no background recording.
Regression	Acquisition/recording/export/replay/map test suites remain green.
Demo	Primary live script and deterministic fallback rehearsed.
Documentation	README, Project Documentation, Master Blueprint, Planning Plan and evidence reports synchronized to RC.

Planning status

PLAN READY FOR EXECUTION. The repository foundation through offline map presentation is already strong. The project is not yet navigation-ready; execution must begin at the frame/calibration gate, while the Python session interoperability layer closes in parallel. The final SIH claim remains blocked until G10 evidence is produced.

Appendix A - Immediate Next Actions

1. Create a clean branch for P0/P1 support work and synchronize current-status documentation without changing verified runtime behavior.
2. Implement the Python exported-session reader/comparison layer so every new physical capture becomes reproducible evidence.
3. Design the P2 calibration protocol and acceptance thresholds before collecting new vehicle data or modifying mechanization.
4. Collect fresh canonical Android device-frame calibration/reference runs using the existing recorder.
5. Only after P2 passes, begin the new classical NavigationEngine mechanization implementation and synthetic truth suite.

Appendix B - Claim Boundaries During Execution

Safe claim	Do not claim until evidence exists
The Android app acquires and records real phone IMU/location data on the tested OnePlus 12R.	That the current app performs real dead reckoning during GNSS loss.
The app exports and replays canonical sessions locally.	That Android recording validates historical IO-VNBD frame semantics.
The app renders an offline Hyderabad map and a synthetic outage/recovery demonstration.	That the synthetic red/purple traces are measured AI/INS results.
The repository has a tested NavigationPresentation adapter and NavigationEngine interface.	That a production NavigationEngine is implemented.
Historical INS results show very large error and motivate a corrected baseline.	That AI can fix the historical INS without resolving frames/mechanization.
Routing/map matching are planned as separate downstream components.	That the current road overlay snaps to roads or computes routes.